

AWS Event-Driven Image Optimization Pipeline Report

Student Name: Dipesh Verma | Date: June 18, 2026 | Architecture: AWS S3 & Lambda (Python / Pillow Layer)

1. SYSTEM INFRASTRUCTURE SETUP

The serverless image processing architecture decouples storage domains by using two distinct, isolated Amazon S3 buckets configured within the identical ap-south-1 (Mumbai) region:

Bucket Identifier	Configured System Specifications & Objectives
Source Bucket dipeshh-images-source	Ingests incoming high-resolution source media files. Configured with strict private access permissions to block unauthenticated public entry.
Destination Bucket dipeshh-images-destination	Receives downscaled, optimized thumbnail images under a dedicated thumbnails/ prefix logical directory structure.

AWS S3 Source Bucket Inventory View

Figure 1: AWS S3 Management Console displaying source bucket containing the uploaded asset 'download.jpg'

2. AUTOMATED PIPELINE DELIVERABLES

Upon a successful file arrival event within the source repository, an ObjectCreated notification pipeline automatically triggers an AWS Lambda worker loop. The runtime layer leverages the Pillow image library to programmatically crop, compress, and scale down the image pixel matrix layout to a standardized 150x150 dynamic boundary size.

AWS S3 Destination Bucket Inventory View

Figure 2: AWS S3 Management Console confirming the automated creation of the 'thumbnails/' folder structure within the destination bucket

3. IMPLEMENTATION CONCLUSION

The operational console evidence validates that the serverless, event-driven components interact exactly as expected. Uploading raw data objects to dipeshh-images-source instantly wakes up the compute environment, executing the thumbnail normalization routine and saving the structural output efficiently to the target directory.